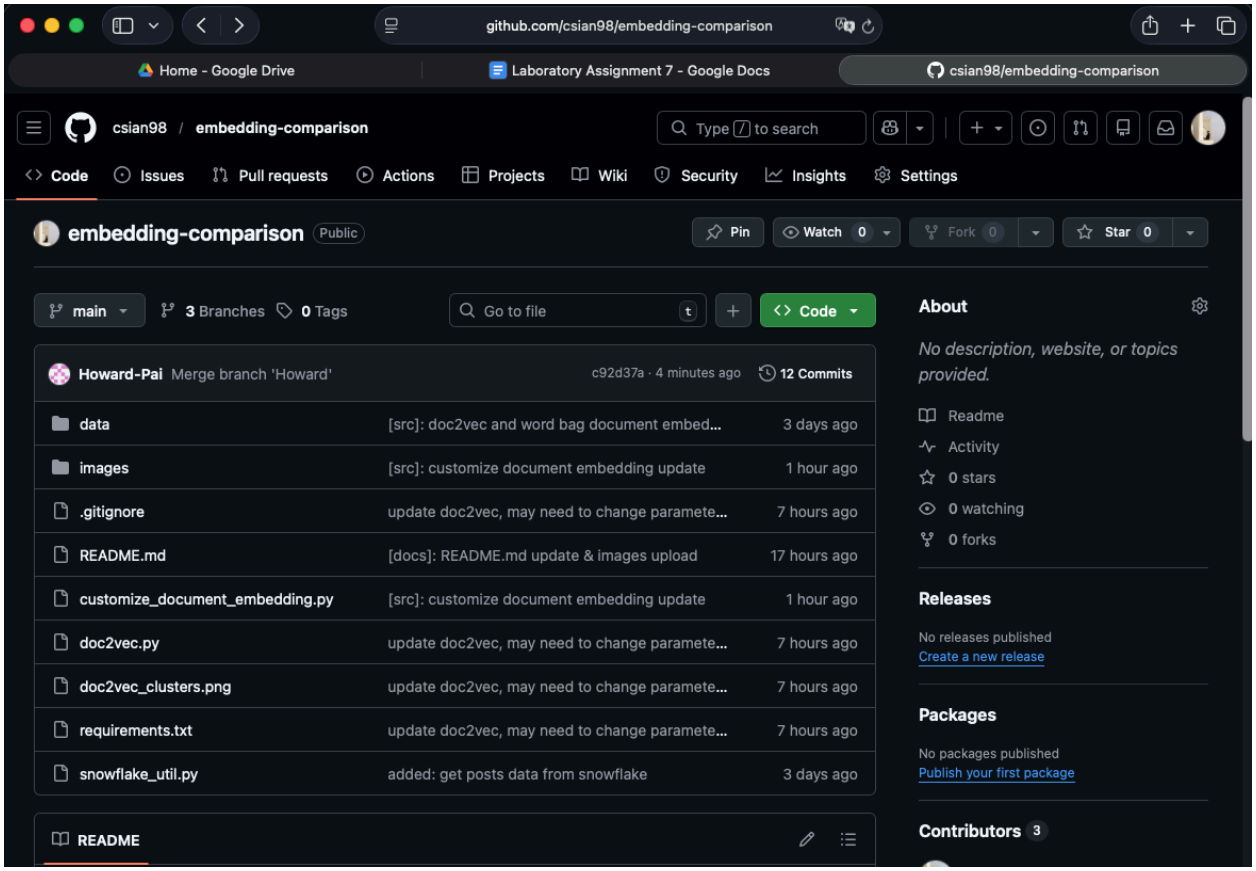
DSCI 560
Data Science Professional Practicum

Laboratory Assignment 8

Group 15: Jeong Hoon Choi, Viraj Bansal, Chih-Yun Pai



Git Hub Repository: https://github.com/csian98/embedding-comparison

Doc2Vec Document Embedding

We will use Reddit data and implement Doc2Vec embedding, then cluster based on that.

- **Data Preparation**

  Before training, each post is tokenised using simple preprocessing: the text is lowercased, punctuation and numbers are stripped, and the result is split into a list of word tokens. Each tokenised post is wrapped in a TaggedDocument object, which pairs the list of tokens with a unique ID corresponding to its row index in the dataset. This ID allows the model to track and update each document's vector independently during training.

```
def build_tagged_docs(df: pd.DataFrame) -> list[TaggedDocument]:
    tagged = []
    for idx, row in df.iterrows():
        text = str(row["SELFTEXT"]) if pd.notna(row["SELFTEXT"]) else ""
        tokens = simple_preprocess(text, deacc=True)
        tagged.append(TaggedDocument(words=tokens, tags=[str(idx)]))
    return tagged
```

- **Configuration settings**

  There are two approaches to embed a document. PV-DM (Distributed Memory) predicts a target word given both its surrounding words and the document vector, meaning word order and local context are taken into account. On the other hand, PV-DBOW(Distributed Bag of Words algorithm) predicts context words from the document vector alone, ignoring word order entirely.

  We can also set the vector size, which is the dimension of the output embedding. A larger vector size can capture richer meaning but requires more data and more training epochs to fill the space meaningfully.

  The three configurations we used are as follows. Config A uses a vector size of 64, PV-DM. Config B uses a vector size of 128, PV-DBOW. Config C uses a vector size of 256, PV-DM. All of them ignore words with a total frequency less than 3 and use 25 epochs. We will cluster the document into 6 groups, which is the actual number of topics in our data. Normally for unsupervised learning we should not inject prior knowledge about the label. But in order to better measure how well embeddings naturally recover the known groupings and compare between different embedding methods, we will fixed to k=6 clusters here.

- Vector Extraction and Cosine Distance

After training, each document is represented by its learned vector. These vectors are then L2-normalised, meaning each vector is scaled so that its length equals exactly 1. This is done because the clustering step uses K-Means, which minimises Euclidean distance. After L2 normalisation, the Euclidean distance between any two vectors becomes mathematically equivalent to their cosine distance.

```
vecs = np.array([model.dv[doc.tags[0]] for doc in tagged_docs])
return normalize(vecs, norm="l2")
```

- Evaluation Metric

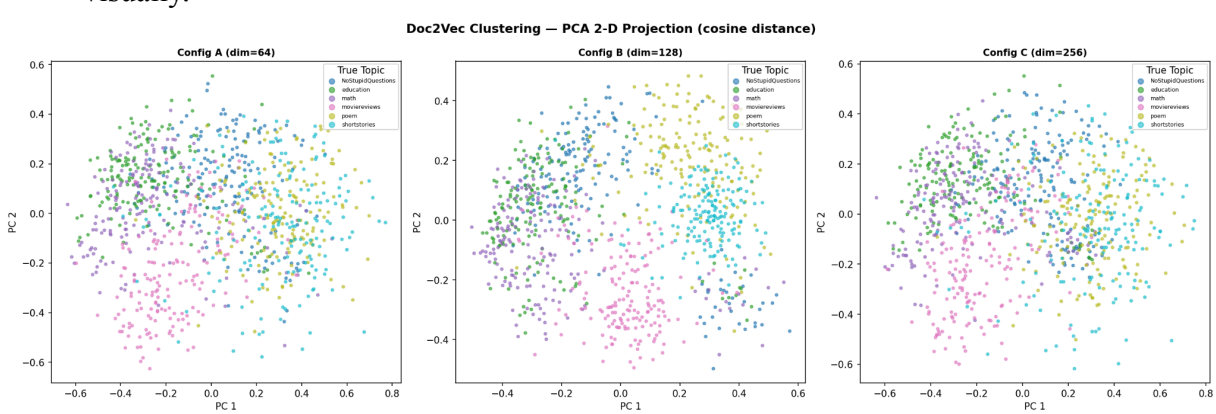
Three metrics are used to evaluate the quality of the clusters produced by each configuration: The silhouette score measures, Davies-Bouldin score and Intra-cluster cosine similarity. A higher silhouette score indicates that documents are well-matched to their assigned cluster and clearly separated from neighbouring clusters. A lower Davies-Bouldin score indicates more compact and well-separated clusters. And finally, a higher similarity value means the documents in each cluster are more semantically consistent with one another.

	silhouette	davies_bouldin	avg_intra_cosine	n_clusters
Config A (dim=64)	0.0978	3.2230	0.3869	6.0
Config B (dim=128)	0.0598	3.5960	0.4074	6.0
Config C (dim=256)	0.0935	3.2769	0.3875	6.0

After clustering, Config A and C are better in forming clusters.

- Visualisation

To visualise the clusters, the high-dimensional document vectors are projected to 2 dimensions using Principal Component Analysis. PCA finds the two directions in the vector space that capture the most variance, allowing the cluster structure to be inspected visually.



Word2Vec & BoW Document Embedding

We used Word2Vec for the words in the Reddit data and clustered them into K categories (bins), and then used the Bag of Word (BoW) method to embed the documents into vectors.

- ## Data Preparation

  We used data collected from six subreddits NoStupidQuestions, Education, Math, MovieReviews, Poem, and Shortstories stored in SnowFlake in the same manner as above. In the data table, the TITLE, SELFTEXT, and TEXT columns were collected for each row and used as one document.

  As with the previous method, stop words were excluded from the analysis.

- ## Documentation Embedding

  To perform vector embedding for a single document, three processes were used: Word2Vec, Vector Clustering, and Bag of Words to create a vector containing the context of the document.

  First, Word2Vec was trained using words from all documents except Stopwords. Among the Continuous Bag of Words (CBOW) and Continuously Sliding Skip-Gram methods, the Skip-Gram method was used to train the model because it allows for more learning with less data. When training the model, the window size of Skip-Gram was fixed to 5. The word embedding sizes using Word2Vec were the same as those of Doc2Vec above 64, 128, and 256.

  K-Means Clustering was used to perform clustering into appropriate bins for all words embedded through Word2Vec. Using clustering, words predicted closely by Word2Vec can be used as labels in a single bin. This allows us to encapsulate documents using a much smaller embedding vector size than previously used with Doc2Vec. We used a clustering count of 8, which will become the final embedding size.

  Finally, the final Document Embedding Vector was generated by calculating the number of words per category contained in each document using the previously labeled results for all words. This method is an improvement on the existing Bag of Words method, which encapsulates the meaning of each document using the number of words per category contained in each document.

- TF-IDF Normalization

The final word vectors obtained from this method were then generated using TF-IDF Normalization. Rather than simply converting the document's category frequency into a unit vector, the weights were calculated by considering the category frequencies in other documents, resulting in better clustering performance.

- Evaluation Metric

Documentation Embedding was performed for a total of three setting values, and K-Means Clustering was performed using the embedded vectors. For the final evaluation, K-Means Clustering and Evaluation Score used the same function as the above method, and Documentation Embedding was performed using the same bins size=8 while changing the word embedding dimensions of Word2Vec to 64, 128, and 256, and the results were compared.

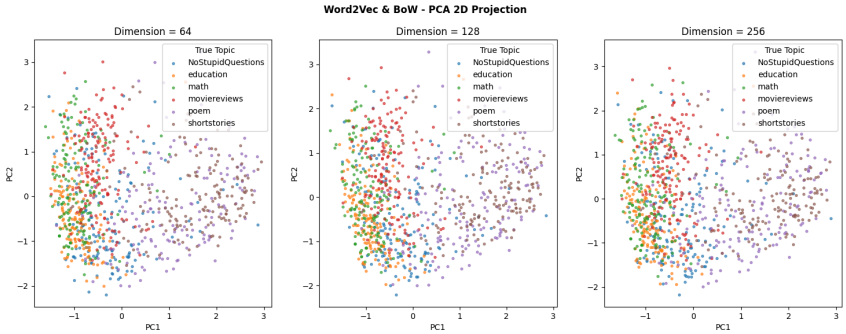
```
~/synology-reliquia/academy/USC/MSDS/DSCI-560/laboratory/lab8/embedding-comparison (main*) » python customize_document_e
mbedding.py
[nltk_data] Downloading package punkt_tab to /Users/csian/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /Users/csian/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!

Fetching posts from Snowflake...
Fetched 996 posts from Snowflake
2026-03-07 20:04:00.294 Python[23980:614356] ApplePersistenceIgnoreState: Existing state will not be touched. New state
will be written to /var/folders/38/r9ht3h492d9_vhmntvyxpbzr0000gn/T/org.python.python.savedState
Metrics: {'silhouette': 0.4031, 'davies_bouldin': 1.2726, 'avg_intra_cosine': 0.865, 'n_clusters': 6}
Metrics: {'silhouette': 0.4061, 'davies_bouldin': 1.2761, 'avg_intra_cosine': 0.864, 'n_clusters': 6}
Metrics: {'silhouette': 0.4033, 'davies_bouldin': 1.2946, 'avg_intra_cosine': 0.8638, 'n_clusters': 6}
```

The output values above are Silhouette, Davies_bouldin, and average Intra Cosine scores of Documentation Embedding using Word2Vec word embedding sizes 64, 128, and 256, respectively.

- Visualisation

For the three settings, the same as Doc2Vec above, PCA was used on the 8-dimensional embedded vector to visualize the two orthogonal dimension vectors with the highest variance, and the true labels were displayed in color.



Document Embedding Comparison

In our setup, we measured the performance of Word2Vec & BoW vs Doc2Vec embedding methodology in recovering the 6 clusters (subreddits). You can observe that the silhouette and Davies Bouldin values we recorded for Word2Vec & BoW for each of the 3 dimensions ranged from 0.403 - 0.406 and 1.273 - 1.295 respectively. Compared to Doc2Vec's 0.0598 - 0.0978 for Silhouette and 3.223 - 3.596 for Davies Bouldin. Since Word2Vec & BoW has higher Silhouette values and lower Davies Bouldin values compared to Doc2Vec, Word2Vec & BoW outperforms Doc2Vec in terms of quantitative metrics. This is also supported by the average intra-cosine similarity metric where the higher values for Word2Vec & BoW indicate that this embedding methodology allows for tighter clustering.

If we graph the 6 clusters after performing PCA, we can see that Word2Vec & BoW offers higher visual cluster separation and fidelity compared to Doc2Vec. In Doc2Vec there is heavy overlap indicating a lower quality recuperation of the original 6 clusters. This is consistent with our intra-cosine similarity metric.

We chose our metrics to help us measure the overall cluster separation and fidelity. Silhouette helps us capture the cohesiveness and separation in one-value of the clusters, while the Davies Bouldin helps emphasize the cluster compactness and separation of the clusters. We also chose to measure the average intra-cosine similarity metric since it is directly aligned with the embedding geometry.

Some observed advantages of Word2Vec & BoW:

- Better cluster separation
- Strong metrics
- Stable behavior across dimension sizes

Some observed disadvantages of Word2Vec & BoW:

- Loses word order and nuanced context
- Still has evidence of slight overlap

Some observed advantages of Doc2Vec:

- Captures document-level directly

Some observed disadvantages of Doc2Vec:

- Poorer clustering quality
- Sensitive to corpus size/domain